

HashiCorp Vault 1.21 as a Self-Hosted Identity Provider: The Complete Production Reference

TL;DR

- **Vault Community Edition 1.21 can do everything you need for free:** act as an OIDC provider issuing JWTs your FastAPI/React apps verify, broker Azure AD/Entra logins, map Azure security groups to least-privilege policies, and authenticate machine services via AppRole — but you must run it with TLS, a real HCL config file, Integrated Storage (Raft), auto-unseal, and at least one audit device. The OIDC provider, JWT/OIDC auth, identity engine, Raft, and auto-unseal are all in the free Community edition; Sentinel EGP/RGP, control groups, namespaces, and step-up/MFA-beyond-login are Enterprise-only.
- **The recommended architecture:** humans authenticate through Vault's OIDC auth method against Azure AD → Vault maps Azure group GUIDs to three Vault external groups (users/staff/admins) via group aliases → each external group carries one least-privilege ACL policy. Apps then trust Vault as their OIDC provider, fetching public keys from Vault's unauthenticated JWKS endpoint. Machines use AppRole with response-wrapped, short-lived, CIDR-bound SecretIDs and batch tokens.
- **Watch the platform shift:** IBM completed its acquisition of HashiCorp on February 27, 2025 for \$6.4 billion; Vault is under the BSL 1.1 (source-available, not OSI open-source) since August 10, 2023; and the line after 1.21 is **Vault 2.0** (IBM lifecycle versioning). A 1.21-era change makes `disable_mlock` mandatory (no default) for Raft and removes the binary's built-in memory-lock capability in containers, so you **must** set `disable_mlock = true` and disable/encrypt swap.

Key Findings

1. Editions and licensing (verify before you build). On August 10, 2023, HashiCorp relicensed Vault from MPL 2.0 to the Business Source License (BSL) 1.1 — source-available, free for almost all uses, but not OSI open-source. IBM completed its acquisition of HashiCorp on February 27, 2025 (\$35/share, \$6.4 billion). The free edition is now called **Vault Community Edition**. Everything in this document targets Community Edition 1.21 (released October 22, 2025) unless explicitly flagged Enterprise. The OpenBao fork (MPL 2.0, Linux Foundation) is API-compatible if BSL is a concern.

2. Community vs Enterprise feature boundary (critical for your free deployment). Available in Community 1.21: OIDC provider feature, JWT/OIDC auth, identity engine (entities/groups/aliases/templated policies), AppRole, Integrated Storage (Raft), auto-unseal (Transit + all cloud KMS), audit devices, ACL policies, Login MFA (TOTP on login). **Enterprise-only:** Sentinel EGP/RGP policies, control groups (step-up dual-control authorization), namespaces

(multi-tenancy), performance/DR replication, Step-up Enterprise MFA, seal wrapping, automated snapshots, HSM/seal-HA.

3. Vault as OIDC provider works in two layers. The simpler **identity token roles**

(`identity/oidc/key`), (`identity/oidc/role`), (`identity/oidc/token`) let an already-authenticated entity request a signed JWT — ideal for machine workloads. The **full OIDC provider** (`identity/oidc/provider`), (`/client`), (`/scope`), (`/assignment`) implements the OAuth2 authorization-code flow with PKCE for browser apps. Both publish unauthenticated discovery/JWKS endpoints your apps verify against.

4. The Azure → Vault → apps chain is fully supported and is your best design for three human roles.

Details

SECTION 1 — Core Concepts & Architecture

Security model & the barrier. Vault encrypts all data at rest with AES-256-GCM behind a cryptographic *barrier*. The barrier is sealed by an *encryption key*, which is itself encrypted by the *root key* (formerly "master key"). Nothing can be read or written until Vault is unsealed.

Seal/unseal. On start, Vault is *sealed*: it has the ciphertext but not the root key. Unsealing reconstructs the root key.

- **Shamir** (default): the root key is split into N key shares with a threshold T (e.g., 5 shares, threshold 3). Operators supply T shares to unseal. Manual on every restart.
- **Auto-unseal**: an external KMS (cloud KMS, or a second Vault's Transit engine) holds the unwrapping key; Vault unseals itself on start. Under auto-unseal, (`vault operator init`) generates **recovery keys** (not unseal keys) — these authorize operations like (`operator generate-root`) and manual reseal, but **cannot themselves decrypt the root key**.

The token is the core auth currency. Every authenticated request carries a token (header (`X-Vault-Token`)). Auth methods exist only to *exchange* some external credential (password, JWT, AppRole SecretID, cloud identity) for a Vault token. Secrets engines and the identity system are then accessed *with* that token.

Secrets engines vs auth methods vs identity.

- *Auth methods* (mounted under (`auth/`)) verify identity and issue tokens.
- *Secrets engines* (mounted under e.g. (`secret/`), (`identity/`), (`transit/`)) store/generate secrets.
- *Identity* ((`identity/`), mounted by default, cannot be disabled/moved) is the internal layer tying logins from different auth mounts to a single canonical *entity*, and grouping entities for shared authorization.

Path-based API. Everything is an HTTP path under `/v1/`. CLI commands map directly: `vault read auth/approle/role/x` → `GET /v1/auth/approle/role/x`. Vault uses a non-standard `LIST` verb (or `GET ?list=true`). 1.21 note: Vault now rejects non-canonical paths (double slashes) and redirects `/./`, `/../`.

Mount points & mount accessors. Each mounted engine/auth method lives at a *path* and has a unique, immutable *mount accessor* (e.g. `auth_oidc_1a2b3c4d`). The accessor is how the identity system pins an alias to a specific auth mount — central to group aliases (Section 7).

Request lifecycle. Client sends request with token → Vault checks token validity → computes *token policies* (set at login) + *identity policies* (from entity + its groups, evaluated at request time) → ACL evaluation (most-specific path wins; explicit `deny` always wins) → if Enterprise, RGP/EGP/MFA → routes to the engine → audit device logs request and response (HMAC'd) → returns response.

SECTION 2 — Auth Methods

Human-facing:

OIDC (`auth/oidc`) — *recommended for humans*. Browser-based SSO against an external IdP (Azure AD, Google, Okta, Keycloak). Vault redirects the user to the IdP, receives the ID token, validates it, maps claims to an entity + groups, issues a Vault token. CLI flow: `vault login -method=oidc` opens a browser, IdP redirects to `http://localhost:8250/oidc/callback`. UI flow redirects to `<VAULT_ADDR>/ui/vault/auth/oidc/oidc/callback`. Both URIs must be in `allowed_redirect_uris`.

userpass (`auth/userpass`) — local username/password. Simple, no external dependency. Good for break-glass/bootstrap accounts and the OIDC-provider end-user demo. User lockout enabled by default. Not ideal as primary human auth at scale.

LDAP (`auth/ldap`) — authenticate against AD/LDAP directory; maps LDAP groups to Vault external groups via group aliases (alias name = the LDAP group DN/CN). User lockout enabled by default.

GitHub (`auth/github`) — authenticate by GitHub org/team membership; team slug maps to a group alias. Convenient but weaker (relies on GitHub PATs).

Machine/service:

AppRole (`auth/approle`) — *the recommended machine auth*. Two credentials: a `RoleID` (like a username, semi-public) and a `SecretID` (like a password, sensitive). Best practice ("pull mode"): RoleID delivered out-of-band, SecretID delivered **response-wrapped** so only the final app can unwrap it. Complete role parameter list with exact defaults (from the AppRole API docs):

- `bind_secret_id` (bool, **true**) — require a SecretID at login.
- `secret_id_bound_cidrs` (array, []) — CIDR blocks allowed to perform login.
- `secret_id_num_uses` (int, **0** = unlimited) — uses before a SecretID expires.
- `secret_id_ttl` (string, "" = no expiry) — SecretID lifetime.
- `token_ttl` (**0**), `token_max_ttl` (**0**), `token_explicit_max_ttl` (**0** — hard cap even over renewal).
- `token_policies` (array, []), `token_bound_cidrs` (array, []) — ties the issued token to source CIDRs.
- `token_num_uses` (int, **0** = unlimited; set 0 if the token must create child tokens).
- `token_period` (**0**) — max period for periodic tokens from this role.
- `token_type` (string, default "" → resolves to mount default `service`; **HashiCorp explicitly recommends `batch` for machine-based authentication**).
- `local_secret_ids` (bool, **false**, set-once at creation) — cluster-local SecretIDs.

Secure delivery: `vault write -wrap-ttl=120s -f auth/approle/role/<r>/secret-id` returns a single-use wrapping token; the app unwraps once. Tamper-evidence: if an attacker unwraps first, the legitimate app gets an error — alert on it. "Pull mode" (Vault generates the SecretID) is preferred over "push mode" because only the final client ever sees the SecretID. HashiCorp Developer

HashiCorp Developer

JWT auth (`auth/jwt`) — validates a pre-obtained JWT (no browser). For CI/CD (GitHub Actions OIDC), Kubernetes projected SA tokens, or any workload presenting a signed JWT. Configure via `jwtks_url`, `oidc_discovery_url`, or static `jwt_validation_pubkeys`. Role type `jwt`.

Kubernetes auth (`auth/kubernetes`) — pods authenticate with their service-account JWT; Vault validates against the cluster's token reviewer/JWKS.

TLS cert auth (`auth/cert`) — mutual-TLS client certificates map to roles.

Cloud auth — AWS (IAM/EC2 instance identity), Azure (MSI — validates `xms_mirid`/`xms_az_rid` claims, resource group/VM name), GCP (IAM/GCE). Best when the workload runs on that cloud, because the platform itself vouches for identity (solves "secret zero").

The "secret zero" problem. Every machine-auth chain bottoms out at one credential the app must already possess. Platform-native methods (Kubernetes/AWS/Azure/GCP) eliminate it because the platform attests identity. AppRole minimizes it via response wrapping + short SecretID TTLs + use limits + CIDR binding, plus a trusted broker (e.g., your orchestrator) that only has permission to fetch *wrapped* SecretIDs.

All auth methods tie into identity: on first successful login Vault auto-creates an *entity alias* on that mount and (if needed) a canonical *entity*; `groups_claim`/LDAP groups feed external-group membership.

SECTION 3 — Identity Secrets Engine (central)

Entities & entity aliases. An *entity* is the canonical representation of a human or machine. An *entity alias* links a specific login on a specific auth mount to that entity. One person with Azure + userpass logins = one entity with two aliases (each on a different mount accessor). Constraint: two aliases on the *same* auth mount cannot map to the same entity. On login, if no alias exists, Vault auto-creates entity + alias. Entity policies are evaluated at *request* time, so changing an entity's groups/policies affects already-issued tokens immediately. (HashiCorp Developer + 2)

Internal vs external groups. *Internal* groups: you list `(member_entity_ids)`/`(member_group_ids)` explicitly. *External* groups: membership is driven by an external IdP. An external group has exactly **one** *group alias* whose `(name)` matches the external group identifier (LDAP DN, GitHub team slug, or **Azure group object GUID**) and whose `(mount_accessor)` points at the auth mount. On login/renewal Vault adds the entity to the external group if the IdP asserts that group. (HashiCorp Developer)

Group aliases & mount accessors. `(vault write identity/group-alias name="<external-group-id>" mount_accessor="<auth_mount_accessor>" canonical_id="<vault_group_id>")`. The mount accessor is what binds the alias to the right auth method. (HashiCorp Developer)

Identity metadata & templated policies. Entities/aliases/groups carry metadata. ACL policies can interpolate it: `{{identity.entity.id}}`, `{{identity.entity.metadata.<key>}}`, `{{identity.entity.aliases.<mount_accessor>.metadata.<key>}}`, `{{identity.groups.names.<group_name>.metadata.<key>}}`, `{{identity.groups.ids.<group_id>.name}}`. **1.21 caveat:** wildcards/globs in the *rendered* output of a template now yield "permission denied." Templating gotcha: per-user metadata on a shared alias can race if the same entity authenticates concurrently — prefer entity.id-based isolation.

Why identity matters: it gives consistent authorization across auth methods. Whether a user logs in via Azure OIDC or userpass, they resolve to one entity with one set of group-derived policies, and audit logs carry the entity_id for a unified trail.

SECTION 4 — Policies & Authorization

ACL syntax. Policies are HCL/JSON path rules with capabilities:

```
hcl

path "secret/data/myapp/*" {
  capabilities = ["read", "list"]
}
```

Capabilities: `create`, `read`, `update`, `patch`, `delete`, `list`, `sudo` (for root-protected paths), `deny` (always wins, overrides any allow). Plus the new `recover` capability (1.20+) for granular secret recovery.

Wildcards. `*` (glob) matches any sequence including further segments, valid only as the final character; `+` matches exactly one path segment and can appear mid-path (e.g. `secret/+/config`).

Parameter constraints. `required_parameters`, `allowed_parameters`, `denied_parameters` (denied wins), plus `min_wrapping_ttl`/`max_wrapping_ttl` to constrain response wrapping per path.
Example AppRole-broker policy:

```
hcl

path "auth/approle/role/+/secret*" {
  capabilities = ["create", "read", "update"]
  min_wrapping_ttl = "100s"
  max_wrapping_ttl = "300s"
}
```

Default & root policies. `default` is attached to nearly all tokens (token self-management, cubbyhole) and cannot be deleted; it grants no secret access. `root` policy = unrestricted; only root tokens carry it (and root tokens bypass ACLs and Sentinel). Write minimal, least-privilege policies — start from deny, grant only what's needed.

Token vs identity policies. Token policies are fixed at login. Identity policies are computed from the entity + groups at request time and *added* to token policies; you can change access on live tokens by editing groups.

OSS vs Enterprise. ACL policies (above) are Community. **Sentinel** EGP (endpoint-governing, path-tied, can enforce on unauthenticated paths like login) and RGP (role-governing, token/entity/group-tied, pathless) are **Enterprise-only**, as are **control groups** (require N approvers before a secret read completes). Don't design your free deployment around them.

HashiCorp Developer

Three role-based policy sets (concrete):

```
hcl
```

```
# policy: app-users – regular end users, own KV space only
path "secret/data/users/{{identity.entity.id}}/*" { capabilities = ["create","read"] }
path "secret/metadata/users/{{identity.entity.id}}/*" { capabilities = ["list","read"] }

# policy: staff – shared team secrets + read app config
path "secret/data/staff/*" { capabilities = ["create","read","update","delete","list"] }
path "secret/data/apps/*" { capabilities = ["read","list"] }

# policy: admins – manage auth, policies, identity (NOT root)
path "sys/auth/*" { capabilities = ["create","read","update","delete","sudo"] }
path "sys/policies/acl/*" { capabilities = ["create","read","update","delete","list"] }
path "identity/*" { capabilities = ["create","read","update","delete","list"] }
path "sys/mounts/*" { capabilities = ["create","read","update","delete","list"] }
path "auth/token/create/root" { capabilities = ["deny"] }
```

SECTION 5 — Vault as an OIDC Provider

Vault issues OIDC/JWT ID tokens your own apps verify. Two mechanisms, both built on the always-on `identity/` engine.

(A) Simpler identity-token roles — for machine workloads that authenticate to Vault then need a portable JWT:

```
bash

vault write identity/oidc/key/app-key algorithm="RS256" rotation_period="24h" verification_ttl="1h"
vault write identity/oidc/role/app-role key="app-key" ttl="1h" template='{"groups":{"groups":{}}}'
vault read identity/oidc/token/app-role # returns signed JWT
```

Discovery: `GET /v1/identity/oidc/.well-known/openid-configuration`; JWKS: `GET /v1/identity/oidc/.well-known/keys` (both unauthenticated). Issuer defaults to `api_addr` + `/v1/identity/oidc`; set it explicitly to a network-reachable hostname. An authenticated introspection endpoint also checks whether the entity is still active. (HashiCorp Developer)

(B) Full OIDC provider — for browser apps doing the authorization-code flow:

- Key:** `identity/oidc/key/<name>` (algorithm RS256/384/512, ES256/384/512, EdDSA; `rotation_period`, `verification_ttl`, `allowed_client_ids`). Force rotation: `vault write -f identity/oidc/key/<name>/rotate`.
- Client:** `identity/oidc/client/<name>` → returns `client_id` + `client_secret`. `client_type` confidential (has secret) or public (PKCE, no secret). `redirect_uris` must exactly match.

`assignments` restrict who can use it (`allow_all` built-in, or your own entity/group list).

`id_token_ttl`/`access_token_ttl` default 24h. (HashiCorp Developer + 2)

- **Scope:** `identity/oidc/scope/<name>` with a Go-template mapping identity info to claims, e.g. `template='{"groups":{{identity.entity.groups.names}}, "email": {{identity.entity.metadata.email}}}'`. The `openid` scope is built-in and immutable. (HashiCorp Developer)
- **Assignment:** `identity/oidc/assignment/<name>` with `entity_ids`/`group_ids`.
- **Provider:** `identity/oidc/provider/<name>` with `issuer`, `allowed_client_ids`, `scopes_supported`. A built-in `default` provider exists per namespace.

Provider discovery: `GET /v1/identity/oidc/provider/<name>/well-known/openid-configuration` and `/.well-known/keys`. Authorization endpoint:

`/ui/vault/identity/oidc/provider/<name>/authorize`; token endpoint

`/v1/identity/oidc/provider/<name>/token`. All clients treated as first-party (no consent screen); PKCE required for public clients, optional for confidential; authorization codes single-use, ~5-minute lifetime. (HashiCorp Developer) (Enclave)

How FastAPI/React verify the token. Frontend (React) runs the auth-code flow (redirect to Vault authorize endpoint, exchange code at token endpoint via your backend). Backend (FastAPI) validates the JWT: fetch JWKS once and cache, verify RS256 signature against the `kid`, then check `iss` (matches the provider issuer URL), `aud` (your `client_id`), and `exp`. Practical Python:

```
python

import jwt
jwks = jwt.PyJWKClient(f"{VAULT_ADDR}/v1/identity/oidc/provider/main/.well-known/keys")
key = jwks.get_signing_key_from_jwt(token)
claims = jwt.decode(token, key.key, algorithms=["RS256", "ES256", "EdDSA"],
                    audience=CLIENT_ID, issuer=f"{VAULT_ADDR}/v1/identity/oidc/provider/main")
```

The app needs only network reachability to the unauthenticated JWKS endpoint — never a Vault token.

When to use which. Use the **full provider** for human browser SSO into your React/FastAPI apps (real OIDC RP libraries work out of the box). Use **identity-token roles** for machine/service workloads that already hold a Vault token and just need a verifiable JWT to present to a peer service.

SECTION 6 — Vault as an OIDC Client / Identity Broker

Configure `auth/oidc` (or `auth/jwt`) to trust an external IdP. Key config params:

- `oidc_discovery_url` (e.g. Azure `https://login.microsoftonline.com/<tenant>/v2.0`), `oidc_client_id`, `oidc_client_secret`, `bound_issuer`, `default_role`, optional `provider_config` (`provider="azure"`, `"google"`, etc.).
- Role params: `role_type` (`oidc` for browser flow, `jwt` for token validation), `user_claim` (e.g. `sub`, `email`), `groups_claim` (e.g. `groups`), `bound_audiences`, `bound_claims` (map of claim→allowed values, restricts who may log in), `oidc_scopes`, `allowed_redirect_uris`, `claim_mappings` (copy claims into entity metadata), `token_policies`, `ttr`.

`jwt` role = simpler (Vault only validates a presented JWT's signature + bound params). `oidc` role = full browser auth-code flow. Same mount can't do both signature-verification methods at once; mount twice if needed.

SECTION 7 — Azure AD → Vault → Apps, End to End

This is the recommended design for three human roles. Yes, Vault sits as a second layer atop Entra ID: users authenticate via Azure, Vault maps Azure security groups to Vault policies, and Vault then issues its own tokens your apps trust.

Step 1 — Register the app in Entra ID. App registration → record Application (client) ID =

`oidc_client_id`; tenant → `oidc_discovery_url = https://login.microsoftonline.com/<tenant>/v2.0`. Add a client secret = `oidc_client_secret`.

Redirect URIs: `<VAULT_ADDR>/ui/vault/auth/oidc/oidc/callback` and

`http://localhost:8250/oidc/callback`. API permissions: Microsoft Graph `GroupMember.Read.All` (and `profile`).

Step 2 — Emit the groups claim. Token configuration → Add groups claim → choose Security groups (or All); ensure it's in the **ID** token. **Azure sends group *object GUIDs*, not names** — this is the key fact. Microsoft Entra ID limits emitted groups to 200 for JWT (150 for SAML); if a user belongs to more, Entra omits the list and emits an *overage* claim — `_claim_names`/`_claim_sources` pointing to a Microsoft Graph endpoint — instructing the application to query Graph for the user's group membership. Vault handles this for the Entra provider via `fetch_groups`/`distributed_claim_access_token` (Entra-specific).

Step 3 — Configure Vault OIDC auth.

```
bash
```

```

vault auth enable oidc
vault write auth/oidc/config \
  oidc_client_id="<app-id>" oidc_client_secret="<secret>" \
  oidc_discovery_url="https://login.microsoftonline.com/<tenant>/v2.0" \
  default_role="azure-default"
vault write auth/oidc/role/azure-default \
  user_claim="sub" groups_claim="groups" \
  allowed_redirect_uris="<VAULT_ADDR>/ui/vault/auth/oidc/oidc/callback,http://localhost" \
  oidc_scopes="profile" token_policies="default" ttl="1h"

```

Step 4 — Create three Vault external groups + group aliases.

```

bash

# get the OIDC mount accessor
ACC=$(vault auth list -format=json | jq -r '["oidc/"].accessor')
# one external group per Azure group, each carrying ONE least-privilege policy
for pair in "vault-users:app-users:<AZURE_GUID_USERS>" \
  "vault-staff:staff:<AZURE_GUID_STAFF>" \
  "vault-admins:admins:<AZURE_GUID_ADMINS>"; do
  IFS=":" read gname pol guid <<< "$pair"
  GUID=$(vault write -format=json identity/group name="$gname" type="external" policies=$pol | jq -r '.data.guid')
  vault write identity/group-alias name="$guid" mount_accessor="$ACC" canonical_id="$gname"
done

```

Critical: the group-alias `(name)` **must be the Azure group object GUID**, not the display name — Azure asserts GUIDs in the claim, so policies won't map if you use names.

Step 5 — Claim/alias mechanics recap. On login: Azure returns the ID token → Vault validates it → `(user_claim)` (sub) becomes the entity alias name (entity auto-created) → each GUID in `(groups_claim)` is matched against group aliases on the OIDC mount accessor → entity joins those external groups → group policies become identity policies on the issued token.

Step 6 — Chain to your apps. Once logged in via Azure, the user (now an entity in `(vault-users)`/`(staff)`/`(admins)`) drives the Vault OIDC *provider* (Section 5B): your React app initiates the auth-code flow against Vault's provider authorize endpoint; because the user already has a Vault session/cookie, Vault issues an ID token whose `(groups)` claim (via your scope template) reflects the Azure-derived Vault groups; FastAPI verifies it against Vault's JWKS. **Full chain: Azure AD authenticates → Vault maps groups → Vault issues app tokens → apps verify against Vault.**

Service vs batch tokens. *Service* tokens (prefix `hvs.`): persisted, renewable, revocable, support accessors/cubbyholes/child tokens, ~96 bytes. *Batch* tokens (prefix `hvb.`): encrypted blobs, not stored, not renewable/listable/revocable, no accessor/cubbyhole, lightweight and high-scale, ~128 bytes. Use batch for high-volume short-lived machine ops (and for AppRole machine auth per HashiCorp). Orphan batch tokens replicate across clusters; non-orphan batch tokens are bound to their cluster.

Hierarchy & orphans. Tokens form a parent-child tree; revoking a parent revokes all children and their leases. *Orphan* tokens (`-orphan`), requires root/sudo) have no parent, so they survive their creator — use for long-running services.

TTLs & max TTLs. Every non-root token has a TTL; on expiry it (and its leases) are revoked. Renewal extends it, capped by: explicit max TTL on the token (hard cap) > mount max TTL (tune) > system max TTL (default **32 days**, configurable). A role/group/user-suggested value may shorten but not exceed these.

Periodic tokens (`-period=<dur>`, root/sudo): TTL = period, reset on each renewal, **no max TTL** — live forever as long as renewed within the period. For long-running services that can't re-auth. Keep the period short.

Token roles & accessors. Token roles (`auth/token/roles/<name>`) standardize creation (`allowed_policies`, `disallowed_policies`, `orphan`, `renewable`, `token_period`, `token_explicit_max_ttl`). A *token accessor* is a handle that can look up, renew, or revoke a token **without** being the token itself — safe to log; use for audit and emergency revocation.

Revocation. `vault token revoke <token>` (revokes children too), `revoke -accessor`, `revoke-prefix` (by mount path), `revoke -mode=orphan` (revoke but orphan children). Use limits: `num_uses` caps how many times a token may be used.

Best practices. Short TTLs + automated renewal beats long-lived tokens. Revoke on shutdown. Track accessors. Use Vault Agent auto-auth for renewal where possible. Never hand humans periodic tokens.

SECTION 9 — Production Storage & High Availability

File backend now → why migrate. The `file` backend is single-node, no HA, and **snapshots are unsupported**. It's fine to keep for a single-node lab, but for production migrate to **Integrated Storage (Raft)**: HashiCorp-supported, HA-capable, replicates data to every node, enables `operator raft snapshot`.

Migration with `operator migrate` (offline, storage-level copy, no decryption):

```
hcl
```

```
# migrate.hcl
storage_source "file" { path = "/vault/file" }
storage_destination "raft" { path = "/vault/raft" node_id = "node_1" }
cluster_addr = "https://vault1:8201"
```

Stop Vault → `vault operator migrate -config=migrate.hcl` → update the server config to use the `raft` stanza → start + unseal. After migration the Raft cluster has one node; join peers.

Raft config & HA.

```
hcl

storage "raft" {
  path      = "/vault/raft"
  node_id   = "node_1"
  retry_join { leader_api_addr = "https://vault1:8200" }
  retry_join { leader_api_addr = "https://vault2:8200" }
  retry_join { leader_api_addr = "https://vault3:8200" }
}
cluster_addr = "https://vault1:8201" # required for raft
```

With `retry_join` configured, followers auto-join; otherwise `vault operator raft join <leader_api_addr>`. Use an **odd number of nodes** (3 for HA surviving 1 failure, 5 surviving 2) to avoid split-brain. `operator raft list-peers`, `remove-peer`, autopilot (`operator raft autopilot set-config` — dead-server cleanup, stabilization). Only ports 8200 (API) and 8201 (cluster/Raft) needed.

Snapshots/backups (Community supports save/restore): `vault operator raft snapshot save raft.snap` and `... restore raft.snap`. Automated snapshots are Enterprise. **Snapshots are the only supported Raft backup method** — never back up disparate disk copies.

SECTION 10 — Auto-Unseal

So you stop manually unsealing on restart. Under any auto-unseal, init produces **recovery keys** instead of unseal keys.

Transit auto-unseal (second Vault as KMS — works in Community; *seal wrapping* needs Enterprise):

```
hcl
```

```

seal "transit" {
  address      = "https://vault-kms:8200"
  token        = "<orphan token>"    # prefer VAULT_TOKEN env var; should be orphan
  key_name     = "autounseal"
  mount_path   = "transit/"
}

```

The unseal Vault needs a `transit/` engine + key; the token needs only `update` on `transit/encrypt/<key>` and `transit/decrypt/<key>`. Use an orphan token so it doesn't die with a parent.

Cloud KMS auto-unseal:

```

hcl

seal "aws kms" { region = "us-east-1" kms_key_id = "<id>" }
seal "azure keyvault" { tenant_id = "..." vault_name = "..." key_name = "..." }
seal "gcp kms" { project = "..." region = "..." key_ring = "..." crypto_key = "..." }

```

Never put cloud credentials in cleartext in the config — use the instance's IAM role/MSI or env vars.

Trade-offs & seal migration. Auto-unseal creates a hard lifecycle dependency: if the KMS key is deleted, the cluster is unrecoverable even from backups — guard it with cloud SCPs/least privilege. Migration Shamir↔auto-unseal: add the new `seal` stanza (mark the old one `disabled = true`), stop Vault, restart, then `vault operator unseal -migrate` supplying the threshold of existing keys on **each** call. Going Shamir→auto-unseal, your Shamir keys become recovery keys. Vault Enterprise seal HA allows 2-3 seals for redundancy (Community does not).

SECTION 11 — Audit & Observability

Why it's mandatory. With no audit device, you have no record of who accessed what. Enable at least one. **Important behavior:** if *all* audit devices fail to write, Vault **blocks the request** (security over availability) — so enable two devices on independent storage so one failing doesn't halt Vault.

Devices. `file` (most reliable, recommended primary), `syslog` (local syslog only — partial), `socket` (TCP/UDP/Unix to a SIEM; can lose an entry on connection loss — don't rely on it alone).

```

bash

```

```
vault audit enable file file_path=/vault/logs/audit.log
vault audit enable -path=syslog-remote syslog tag=vault facility=LOCAL0
```

HMAC of sensitive values. By default Vault writes HMAC-SHA256 of string values (tokens, secrets) — never plaintext. `log_raw=false` is the default; **never set `log_raw=true` in production.** `hmac_accessor=true` by default. To match a known value against logs, use `sys/audit-hash`. Non-string values (ints/booleans) aren't hashed — wrap sensitive numbers in quotes. `elide_list_responses=true` shrinks huge LIST records. Secure the log: `chmod 0600`, ship/rotate off-box, generate checksums.

Telemetry. The `telemetry` stanza emits metrics to Prometheus/StatsD/DogStatsD/etc. Set `prometheus_retention_time` and optionally `unauthenticated_metrics_access` on the listener. Server (operational) logs differ from audit logs.

SECTION 12 — End-to-End TLS

```
hcl

listener "tcp" {
  address           = "0.0.0.0:8200"
  cluster_address   = "0.0.0.0:8201"
  tls_cert_file     = "/vault/tls/vault.crt"    # leaf + CA chain concatenated, leaf first
  tls_key_file      = "/vault/tls/vault.key"
  tls_min_version   = "tls12"                  # or "tls13" if all clients support it
  # mTLS (optional): require client certs
  tls_client_ca_file = "/vault/tls/client-ca.crt"
  # tls_require_and_verify_client_cert = "true"
}
```

Vault assumes TLS by default; you must explicitly `tls_disable` to turn it off (don't, in prod). Default min is TLS 1.2; TLS 1.3 doesn't accept explicit cipher lists (Go picks Mozilla-modern ciphers). For TLS 1.2 with custom ciphers, set both `tls_min_version`/`tls_max_version="tls12"` and `tls_cipher_suites` (exclude 3DES to avoid Sweet32/CVE-2016-2183). `tls_disable_client_certs` and `tls_require_and_verify_client_cert` are mutually exclusive. Certs reload on SIGHUP. **End-to-end:** if a load balancer/reverse proxy fronts Vault it should **not** terminate TLS — keep traffic encrypted all the way to Vault. Run an internal CA (or `vault pki`) for issuing the server cert; the SAN must match the hostname clients use (and the `api_addr`/issuer hostname). In Docker, mount the cert/key read-only and set `VAULT_CACERT` so the CLI inside trusts the cert. [GitHub + 3](#)

SECTION 13 — Docker / Compose Hardening

Your current compose (vault 1.21, `cap_add: IPC_LOCK`, `user 100:100`, file storage bind mount, custom network) is a reasonable base, but the bare `command: server` won't work for production: `server` needs a config file (`-config=`). The Docker image loads any HCL/JSON in `/vault/config`, or you can pass `VAULT_LOCAL_CONFIG`. (Docker Hub)

1.21-era mlock/IPC_LOCK change (important and verified against primary docs).

`disable_mlock` no longer has a default and is **required** when using integrated storage — per HashiCorp's config reference: *"You must set an explicit value for `disable_mlock` if you use integrated storage."* Vault will fail to start otherwise. Set `disable_mlock = true` for Raft because mlock interacts badly with BoltDB's memory-mapped files. On containers, HashiCorp's CHANGELOG states it removed the `cap_ipc_lock` capability from the binary at build time so Vault runs in common container runtimes, and that *"Operators should set `disable_mlock = true`"* and disable swapping. The "Important changes" doc first stated the image requires IPC_LOCK at build time (*"As we now run the Vault container as an unprivileged user we have to set it at build time"*) and a subsequent patch entry says *"Removed the IPC_LOCK capability for official OCI container images while preserving the non-root vault user runtime model."* Net guidance across both: set `disable_mlock = true` and **disable or encrypt host swap**. The 1.21 image runs as a non-root (`vault`, UID 100) user. (If you see `unable to set CAP_SETFCAP effective capability`, run as root or set `SKIP_SETCAP=1` — the former is not recommended.) (HashiCorp Developer)

Complete production server config (`vault.hcl`):

```
hcl
```

```

ui                = true
disable_mlock     = true
cluster_name      = "vault-prod"
log_level         = "info"

storage "raft" {
    path      = "/vault/raft"
    node_id   = "node_1"
    retry_join { leader_api_addr = "https://vault1:8200" }
}

listener "tcp" {
    address           = "0.0.0.0:8200"
    cluster_address   = "0.0.0.0:8201"
    tls_cert_file     = "/vault/tls/vault.crt"
    tls_key_file      = "/vault/tls/vault.key"
    tls_min_version   = "tls12"
}

seal "transit" {                                # or awskms/azurekeyvault/gcpckms
    address          = "https://vault-kms:8200"
    key_name         = "autounseal"
    mount_path       = "transit/"
}

api_addr          = "https://vault.example.com:8200"
cluster_addr      = "https://vault1:8201"

telemetry {
    prometheus_retention_time = "30s"
    disable_hostname          = true
}

```

Compose hardening checklist. Keep `cap_add: [IPC_LOCK]` only if you keep mlock (otherwise `disable_mlock=true` + drop it); run as non-root (UID 100 — the image default); mount config and TLS **read-only** (`:ro`); persist `/vault/raft` and `/vault/logs` on named volumes/bind mounts; expose **only** 8200 (and 8201 if clustering) on a private network; set `VAULT_ADDR=https://...` and `VAULT_CACERT`; do **not** pass the root token or unseal/recovery keys via env; consider `read_only: true` root filesystem with tmpfs for scratch; disable host swap; set `restart: unless-stopped`. Follow HashiCorp's official Production Hardening guide.

SECTION 14 — Security Hardening Throughout

From HashiCorp's official production hardening guidance (baseline + extended):

- **Don't run as root**; unprivileged service account with no write access to the Vault binary/config. (The 1.21 container already runs as UID 100.)
 - **Root token**: use only for initial setup, then **revoke it** (`vault token revoke <root>`); regenerate on demand with `vault operator generate-root` using recovery/unseal keys. **1.21 change**: `sys/generate-root`, `sys/rekey`, and DR operation-token endpoints are now **authenticated by default** (set `enable_unauthenticated_access` to revert — don't). ([GitHub](#))
 - **Least privilege everywhere**; minimal policies, separate user/app policies, limit wildcards, prefer templated entity-id isolation.
 - **Secret zero / response wrapping**: deliver SecretIDs response-wrapped, short TTL, use-limited, CIDR-bound; deliver RoleID and SecretID by separate channels; remove wrapping token after use; alert on unwrap/auth errors.
 - **End-to-end TLS** (Section 12); HSTS via custom response headers; TLS 1.3 where possible.
 - **Disable swap and core dumps** (sensitive data lives in memory); especially with Raft.
 - **At least one audit device** (two on independent storage); HMAC on; ship to SIEM.
 - **Restrict storage access**; data is encrypted at rest but limit FS access anyway.
 - **Single tenancy** (Vault is the only main process on the host); no direct `docker exec`/SSH — use a hardened jump box.
 - **Synchronize clocks** (NTP) — TTLs and cert dates depend on it.
 - **Upgrade frequently**; subscribe to HashiCorp announcements; watch the CHANGELOG.
 - **Off-boarding**: removing an IdP account doesn't instantly revoke tokens — remove from groups, revoke leases, delete the canonical entity (and disable auth methods to revoke all their tokens).
 - **TTLs/lease management**: smallest workable default/max lease TTLs to bound exposure and reduce storage GC pressure; tune at mount and role level.
-

SECTION 15 — Recommended End-to-End Setup (your scenario)

Target architecture.

1. **Storage**: Integrated Storage (Raft), single node now (`migrate file → raft` with `operator migrate`), grow to 3 nodes when you need HA.
2. **Auto-unseal**: stand up a second small Vault with Transit (or use a cloud KMS) so the main node self-unseals. Save recovery keys offline (PGP-encrypt them).
3. **TLS**: real cert (internal CA or `vault pki`), `tls_min_version=tls12`, mounted read-only in the container; `api_addr`/issuer = your public hostname.

4. **Audit:** `file` device to a persisted `/vault/logs` volume + a second device; HMAC on.
5. **Humans via Azure AD OIDC** → three external groups (`vault-users`/`vault-staff`/`vault-admins`) keyed by Azure group GUIDs via group aliases on the OIDC mount accessor → three least-privilege ACL policies (Section 4).
6. **Machines via AppRole:** per-service role, batch tokens, response-wrapped CIDR-bound SecretIDs with short `secret_id_ttl` and `secret_id_num_uses`; short `token_ttl` with renewal.
7. **Vault as OIDC provider to your apps:** one signing key (RS256, 24h rotation), one provider, one confidential client per app (React SPA can use a public/PKCE client), a `groups`/`email` scope template; FastAPI verifies JWTs against the unauthenticated JWKS endpoint (`iss`/`aud`/`exp`).

Bootstrap command sequence (post-init/unseal):

```
bash
```

0. login with root (revoke at the end)

```
export VAULT_ADDR=https://vault.example.com:8200 VAULT_CACERT=/path/ca.crt
vault login <root-token>
```

1. audit + kv

```
vault audit enable file file_path=/vault/logs/audit.log
vault secrets enable -version=2 -path=secret kv
```

2. policies

```
vault policy write app-users app-users.hcl
vault policy write staff staff.hcl
vault policy write admins admins.hcl
```

3. Azure OIDC (Section 7)

```
vault auth enable oidc
vault write auth/oidc/config oidc_client_id=... oidc_client_secret=... \
  oidc_discovery_url="https://login.microsoftonline.com/<tenant>/v2.0" default_role="
vault write auth/oidc/role/azure-default user_claim="sub" groups_claim="groups" \
  allowed_redirect_uris="$VAULT_ADDR/ui/vault/auth/oidc/oidc/callback,http://localhost
  oidc_scopes="profile" token_policies="default" ttl="1h"
```

```
ACC=$(vault auth list -format=json | jq -r '["oidc/"].accessor')
```

create 3 external groups + GUID group-aliases (loop from Section 7)

4. AppRole for a service

```
vault auth enable approle
vault write auth/approle/role/api-svc token_type=batch token_policies="app-users" \
  token_ttl=20m token_max_ttl=1h secret_id_ttl=10m secret_id_num_uses=1 \
  secret_id_bound_cids="10.0.0.0/24" token_bound_cids="10.0.0.0/24"
RID=$(vault read -field=role_id auth/approle/role/api-svc/role-id)
vault write -wrap-ttl=120s -f auth/approle/role/api-svc/secret-id # deliver wrapping
```

5. Vault as OIDC provider for your apps

```
vault write identity/oidc/key/app-key algorithm="RS256" rotation_period="24h" verification
vault write identity/oidc/scope/groups template='{"groups":[{"identity.entity.groups.name
CID=$(vault write -format=json identity/oidc/client/webapp key="app-key" \
  redirect_uris="https://app.example.com/auth/callback" assignments="allow_all" \
  id_token_ttl="1h" access_token_ttl="1h" | jq -r .data.client_id)
vault write identity/oidc/provider/main issuer="https://vault.example.com" \
  allowed_client_ids="$CID" scopes_supported="groups"
```

6. revoke root

```
vault token revoke <root-token>
```

Recommendations

1. **Immediately (before any real use):** Add a real HCL config file with TLS enabled and `(disable_mlock = true)`; you cannot run production on the bare `(server)` command or on plaintext HTTP. Enable one `(file)` audit device. Migrate `(file)`→Raft. Revoke the root token after setup and store recovery/unseal keys offline (PGP). *Threshold to proceed:* `(vault status)` shows `(Storage Type: raft)`, `(Seal Type)` is your KMS/transit, and TLS is on.
2. **Week one:** Wire Azure AD OIDC and the three external groups; verify a test login returns the expected `(identity_policies)`. Stand up Transit or cloud-KMS auto-unseal so restarts don't block on humans. Add the second audit device.
3. **Before exposing apps:** Configure the OIDC provider, register one client per app, and verify FastAPI validates `(iss)`/`(aud)`/`(exp)` against the JWKS endpoint. Use a public/PKCE client for the React SPA, confidential clients for server-side apps.
4. **For machines:** Use AppRole with batch tokens, response-wrapped SecretIDs (`(-wrap-ttl)`), short `(secret_id_ttl)`/`(secret_id_num_uses=1)`, and CIDR binding. Prefer cloud/Kubernetes native auth if a service runs on that platform (eliminates secret zero).
5. **Scale/HA trigger:** When Vault downtime would halt deployments, move to a 3-node Raft cluster behind a load balancer that does **not** terminate TLS, with autopilot dead-server cleanup. Re-evaluate at 5 nodes if you need to survive two simultaneous failures.
6. **Don't design around Enterprise features** (Sentinel, control groups, namespaces, replication, step-up MFA). If you later need policy-as-code guardrails, dual-control approvals, or multi-tenancy, that's the upgrade trigger — or evaluate OpenBao (namespaces are free there).

Caveats

- **Version/branding flux:** Vault 1.21 (Community + Enterprise) released **October 22, 2025**; the next line is **Vault 2.0** under IBM lifecycle versioning. HashiCorp's "Important changes" doc is now on a v2.x track, and some mlock/IPC_LOCK changes are tagged to "2.0.1/2.0.2" — treat those as the same release era as 1.21. The IPC_LOCK container requirement was *added at build time* and then *removed* in consecutive patch docs; the consistent operator guidance is set `(disable_mlock = true)` and **disable/encrypt swap**. Verify current behavior against your exact 1.21.x patch. `(InfoQ)`
- **Licensing:** Vault has been BSL 1.1 (source-available, not OSI open-source) since August 10, 2023, and IBM-owned since February 27, 2025; Community Edition remains free for your self-hosted use, but confirm the BSL terms suit your projects. Enterprise feature boundaries can shift between releases — verify before relying on any feature being free.
- **Azure groups:** the emit limit is 200 groups for JWT/OIDC (150 for SAML); above that Azure sends an overage `(claim_names)`/`(claim_sources)` distributed claim pointing to Microsoft Graph rather than the GUID list — requiring Graph fetch config

((`fetch_groups`)/(`distributed_claim_access_token`)). Group membership changes in Azure only reflect in Vault on the next login/renewal.

- **ACL template wildcards:** as of recent 1.x, wildcards/globs in *rendered* template output are denied; and shared-alias metadata can race under concurrent logins — isolate by `identity.entity.id`.
- **Auto-unseal lifecycle risk:** deleting the KMS/transit key makes the cluster unrecoverable even from backups. Guard the key with strict IAM/SCPs.
- Some specifics (exact cipher lists, telemetry keys, Graph permission names) should be confirmed against the 1.21 docs for your patch level, as defaults occasionally change between releases.